

General ML packages

Pytorch

PyTorch is an open-source machine learning library developed by Meta (Facebook) AI Research.

Think of it as **"NumPy, but for ML."**

- If we know how to write `np.array([1, 2, 3])` in NumPy, we already know 80% of PyTorch. we just write `torch.tensor([1, 2, 3])`.
- The difference is that PyTorch can move that math onto our graphics card (GPU) to make it 100x faster, and it automatically calculates the calculus (gradients) needed for optimization.

New Standard

Yes, specifically for Research and Science.

- **TensorFlow (Google):** Launched first (2015). It was the industry standard for years. It is robust, heavy, and geared towards **production** (deploying apps to millions of phones). However, it is notorious for being difficult to debug and having a steep learning curve.
- **PyTorch (Meta):** Launched later (2016). It became the "rebel" choice that researchers loved because it felt like writing normal Python.

The "War" is effectively over in the research world:

- In 2018, most academic papers used TensorFlow.
- By 2025, over **80-90% of new research papers** (including the fusion/physics ML field) use PyTorch.
- TensorFlow is still used heavily in legacy corporate environments, but **PyTorch is the engine of modern innovation** (e.g., ChatGPT, Stable Diffusion, and Tesla Autopilot are largely PyTorch-based).

Bayesian Optimisation Packages

1. Scikit-learn (**sklearn**)

Analogy: Think of this as a **General Contractor's Toolbox**. It has a hammer, a saw, a drill, and a wrench. It is designed to do a little bit of everything in Machine Learning (classification, regression, clustering), but it isn't specialized for any one complex task.

- **What it is used for:** It is the industry standard for "classic" machine learning. If we want to predict house prices based on square footage, or classify emails as spam, we use this.
- **Does it have what we need?**
 - **GPR: Yes.** It has a solid **GaussianProcessRegressor**.
 - **Acquisition Functions: No.** It predicts values (Mean/Variance), but it does *not* contain the code to decide "where to sample next" (Probability of Improvement, etc.). we would have to write this math ourself.
 - **Multi-Objective BO: No.** It can predict multiple outputs (e.g., tell us the TBR and Heating), but it has no logic to balance them or find trade-offs.
 - **Pareto Fronts: No.**

Verdict: Great for learning how a Gaussian Process works on a static dataset, but **bad for building an automated optimization loop.**

BoTorch (and Ax)

Analogy: Think of this as a **Formula 1 Engineering Team**. It is highly specialized, expensive to run (computationally), and designed for one specific goal: optimizing complex systems where every test run is expensive.

- **What it is used for:** It is used by Facebook/Meta to tune their huge server algorithms, and by scientists to discover new drugs or materials. It is built specifically for **Bayesian Optimization**.
- **Does it have what we need?**
 - **GPR: Yes.** (It uses **GPyTorch**, which is a more advanced version of GPR that runs on GPUs).
 - **Acquisition Functions: Yes.** It has pre-written code for Expected Improvement (EI), Upper Confidence Bound (UCB), and many others.
 - **Multi-Objective BO: Yes.** This is its superpower. It has specific functions (like **qEHVI**) designed to optimize TBR and Heating simultaneously.
 - **Pareto Fronts: Yes.** It has utility functions to automatically calculate and extract the Pareto Front from our data.

Verdict: This is the correct tool for our Semester 2 goals.

Hierarchy (Pytorch → Botorch → Ax)

To understand the relationship between **Ax** and **BoTorch**, it helps to look at the entire "stack" of software we are about to install.

Think of it like the layers of a car:

1. **PyTorch (The Fuel & Pistons):** The raw computing power. It handles the matrices and runs calculations on the GPU.

2. **BoTorch (The Engine):** The mechanical engineering. It uses PyTorch to build Gaussian Processes and Acquisition Functions. It is pure math.
3. **Ax (The Steering Wheel & Dashboard):** The user interface. It manages the experiment, saves our data, and decides what to do next.

BoTorch (The "Mathematician")

BoTorch provides the **intelligence**. It is a library for **Bayesian Optimization research**.

- **What it does:** It defines the GPR models (Posterior) and the Acquisition Functions (Expected Improvement, UCB).
- **How we use it:** If we use raw BoTorch, we have to write the optimization loop ourself. we have to manually feed data into the model, tell it to fit, ask it to generate a candidate, and then run our simulation.
- **Complexity:** High. It requires understanding tensors and gradients.

Ax (The "Project Manager")

Ax is a platform for **Adaptive Experimentation**. It wraps around BoTorch to make it usable for engineers.

- **What it does:** It handles the **logistics**.
 - It defines the "Search Space" (e.g., Blanket Thickness: 100mm to 800mm).
 - It stores our data (so we don't lose results if our computer crashes).
 - It calls BoTorch in the background to get the next best point.
- **How we use it:** we simply tell Ax: "Here is my function (the reactor simulation), and here are my parameters. Go optimize it."
- **Complexity:** Low. It feels like standard Python scripting.

Which one will we write code in?

we will write 95% of our code in Ax.

we only need to "drop down" into BoTorch if we want to do something extremely mathematically custom (like inventing a brand new type of Acquisition Function that has never been used before). For our semester project, Ax's standard calls to BoTorch will be sufficient.

Why Ax is better for our specific project:

1. **Database Management:** Ax can save our experiment to a JSON or SQL file automatically. If our OpenMC simulation crashes on the 50th run, Ax remembers the first 49 runs so we don't have to restart from zero. BoTorch does not do this.
2. **Service API:** Ax allows for an asynchronous loop. we can ask Ax for a parameter set, go run our OpenMC simulation (which takes time), and then come back and report the result.